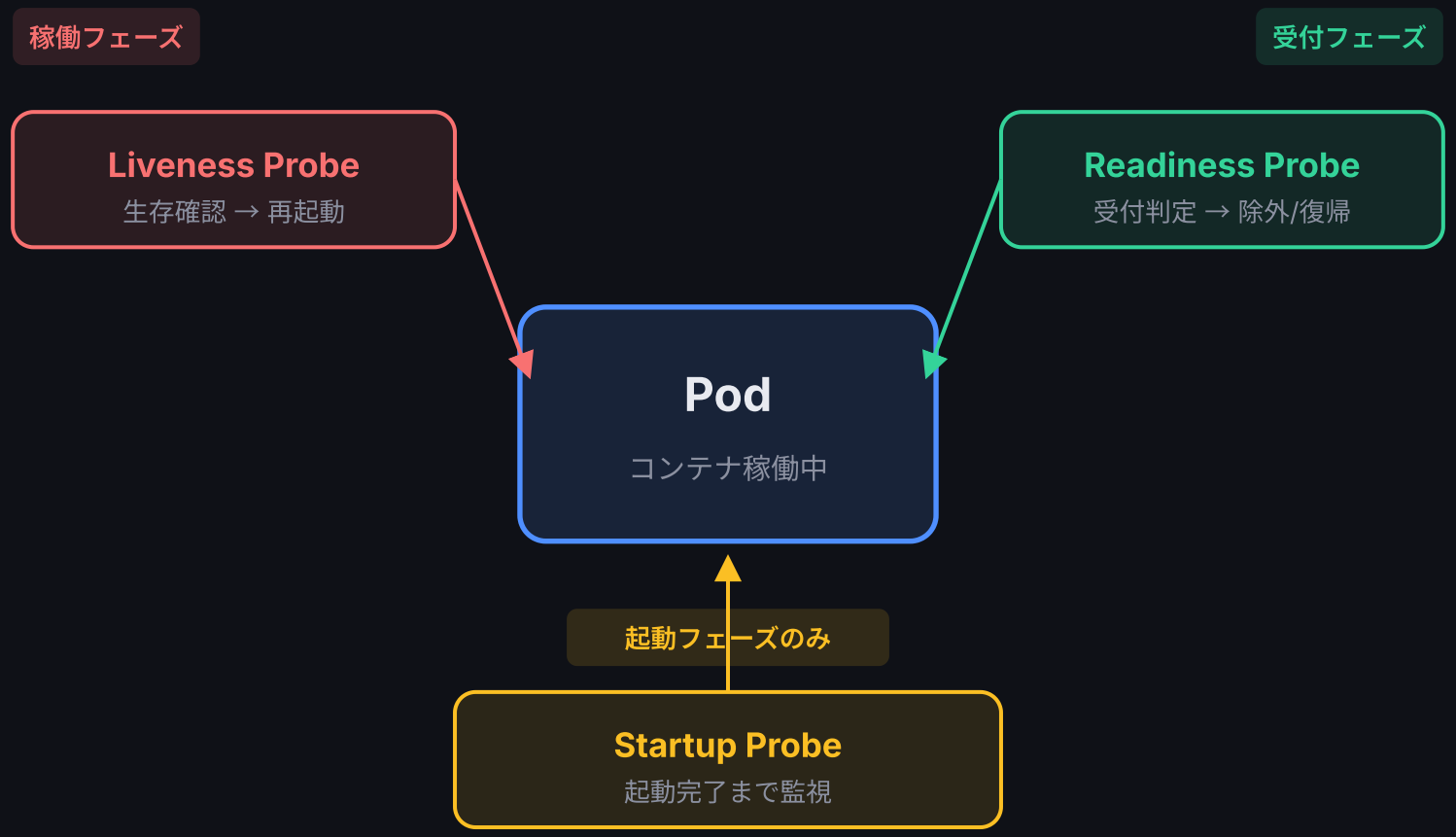
Kubernetes Probe

Probe で
コンテナの異常を自動検知する

Readiness / Liveness / Startup — 3種の使い分けで障害を防ぐ

viserhaut — 2026

3種のProbeが起動・稼働・受付の3レイヤーを守る



Startup Probe

起動完了までLivenessを止める。起動が遅いアプリの**誤再起動を防ぐ**

Liveness Probe

プロセスが応答しなくなったら**自動再起動**。デッドロックを自律回復

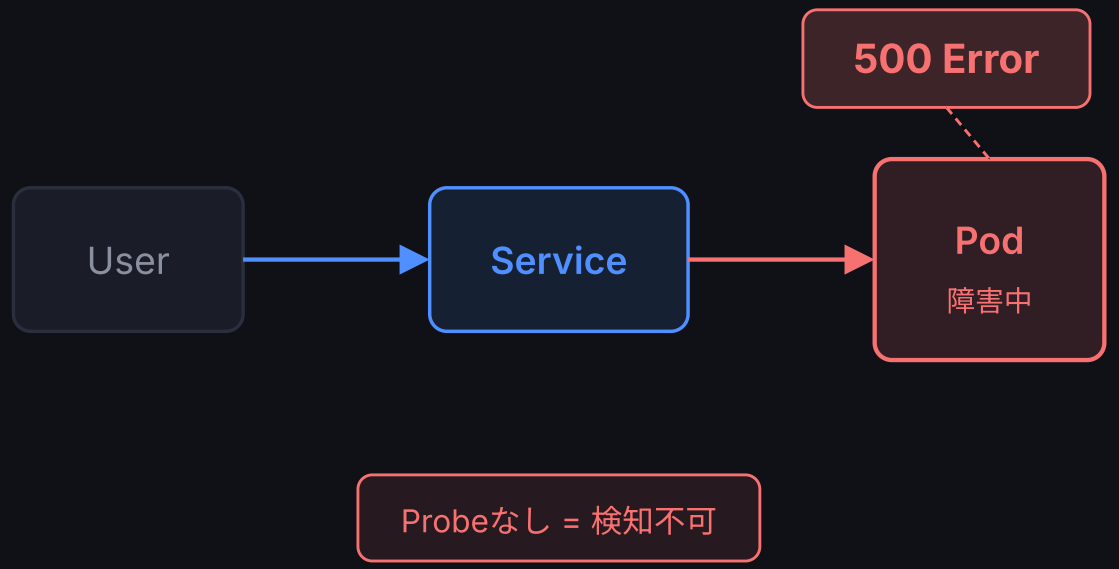
Readiness Probe

受付不可のPodを**Serviceから自動除外**。ユーザーへのエラーを防ぐ

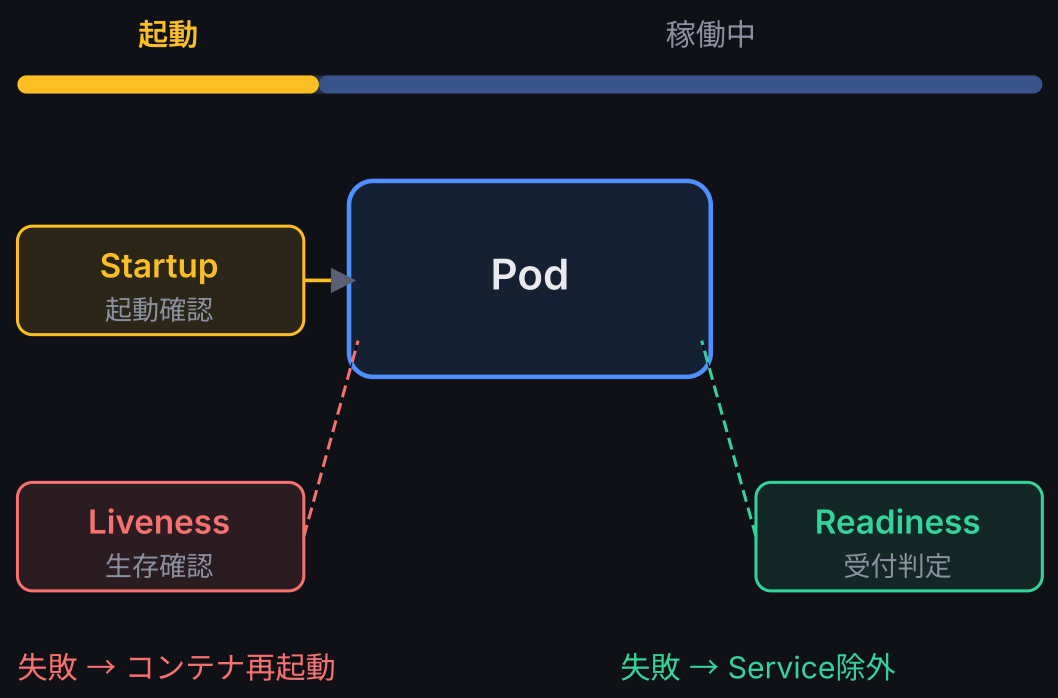
Probeなしではコンテナ障害がユーザーに直撃する

- **起動途中のPodにリクエストが届く**ため、初期化未完了のまま500エラーを返す
- デッドロックが発生しても**Kubernetesは異常を検知できず**、壊れたPodが稼働し続ける
- 再起動後も**準備完了前にトラフィックが流入**し、一時的な障害が長引く

Probeがない状態ではKubernetesの自律回復機能は機能しない



3種のProbeがそれぞれ異なるレイヤーを守る



- **Startup Probe** — 起動フェーズ専用。完了後はLiveness/Readinessに引き継ぎ、**誤再起動を防ぐ**
- **Liveness Probe** — 稼働中ずっと監視。失敗すると**コンテナを再起動**し自律回復を実現
- **Readiness Probe** — 受付能力を判定。失敗するとServiceのエンドポイントから**自動除外・復帰**

Readiness vs Liveness vs Startup — 判断基準の比較

観点	READINESS PROBE	LIVENESS PROBE	STARTUP PROBE
役割	リクエスト受付の準備確認	プロセスの生存確認	起動完了の確認
失敗時の動作	Serviceエンドポイントから除外	コンテナを再起動	Liveness/Readinessを停止し続ける
監視タイミング	稼働中ずっと	稼働中ずっと	起動フェーズのみ
主なユースケース	DB接続待ち・高負荷時の切り離し	デッドロック・無限ループの回復	起動が遅いアプリの誤再起動防止

Probe設定マニフェスト例

```yaml
startupProbe:
  httpGet:
    path: /healthz
    port: 8080
  failureThreshold: 30    # 最大30×10s=5分待つ
  periodSeconds: 10

livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 10
  failureThreshold: 3

readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 5
  failureThreshold: 2
```

startupProbe
failureThreshold×periodSecondsが猶予時間。起動が遅いアプリには大きめに設定

livenessProbe
パスは/healthz（DB接続不要のpingエンドポイント）。失敗=強制再起動なので保守的に

readinessProbe
パスは/ready（DB疎通など実際の受付能力を確認）。Livenessより短い間隔で細かく監視

Pod起動〜障害〜自動復旧のフロー

本番Probe設計3原則

01

Startup Probeで起動猶予を確保する

failureThreshold × periodSecondsが猶予時間。起動に30秒かかるなら最低60秒以上を設定し誤再起動を防ぐ

02

LivenessはDB接続を含めない

DBが落ちてもコンテナを再起動しても意味がない。Livenessはアプリ自身の死活のみを確認するエンドポイントに限定する

03

ReadinessはLivenessより厳密に

短いperiodSecondsと小さいfailureThresholdで細かく判定。一時的な過負荷をすばやく検知しユーザーへのエラーを最小化する